

Automotive Electronic Integration AuE-6200

Project Report

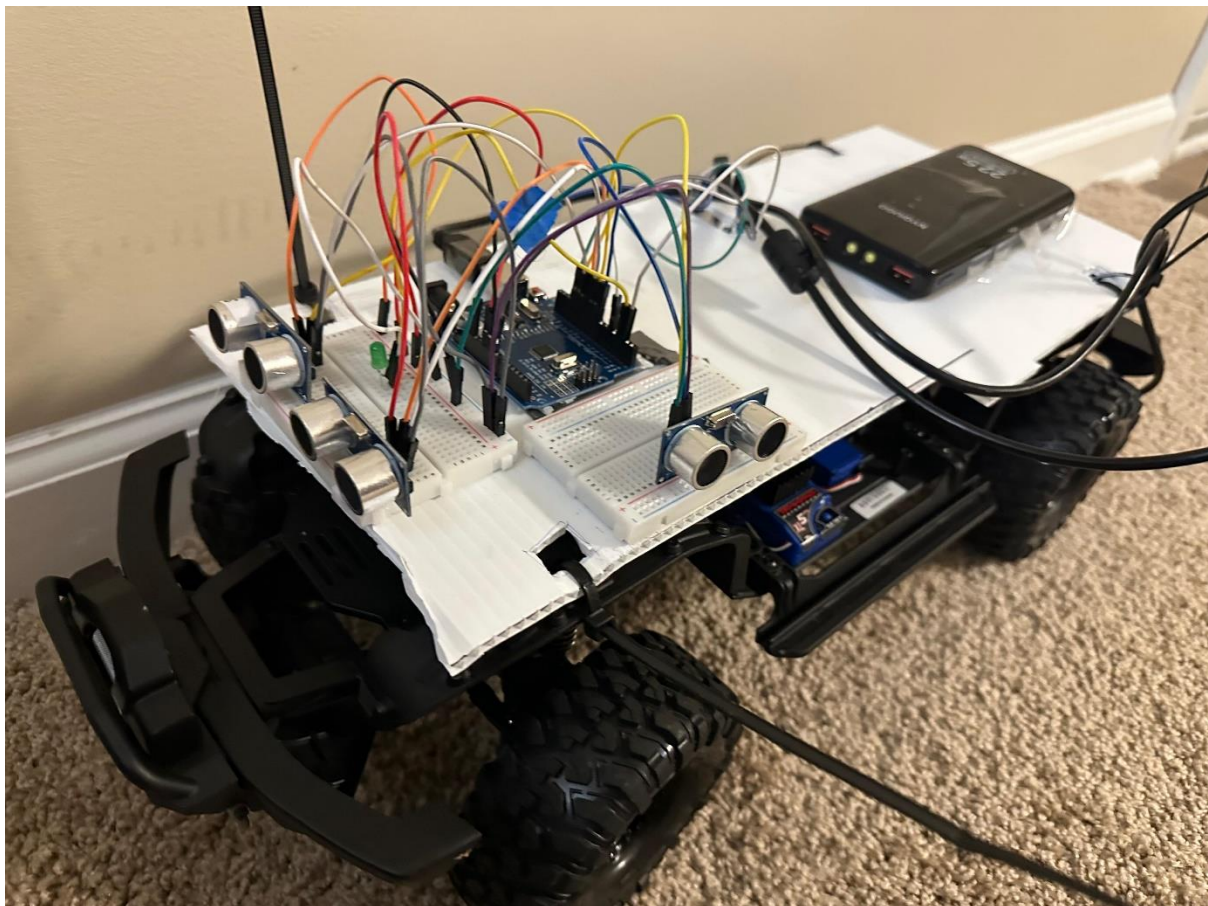
Name - Shantanu Amit More

CU ID - C44539544

Integrity Statement:

I certify that all the work and writing that I contributed to here is my own and not acquired from external sources. I have cited sources appropriately and paraphrased correctly. I have not shared my writing with other students, nor have I acquired any written portion of this document from past or present students.

Shantanu



Project 1

1. Signal Processing for One Ultrasonic Sensor

1.1 Problem Statement

Project 1.1 test consisted of measuring the distance of an object as accurately as possible. The distance range was **1.5 m to 0.2 m**. The data from the sensors was required to be sampled, calibrated and filtered. Both accuracy and time cost were to be optimized. The microprocessor used for this test was Arduino UNO. The code was required to stop when the measurement criteria were met. The sensor was to be mounted in the front part of an RC vehicle.

1.2 Technical Approach

1.2.1 Data Sampling – The sampling rate was optimized to get maximum accuracy. The time sequence was that the sensor would be given a ‘HIGH’ input for 10 microseconds, then a ‘LOW’ signal. After a **delay of 75 milliseconds and 2 microseconds** the sensor is triggered again. The sensor waits for **1200000 microseconds** for recording the pulse after it is sent the ‘LOW’ signal. This ensures that the sensor will record pulse even for a long-distance obstacle. Effectively our sampling time was almost **equal to 75 milliseconds**.

1.2.2 Data Filtering – The time data sampled from the sensor had to be filtered for noise. We used Kalman filter for filtering the data. The Kalman filter equation was as follows

Estimation

$$\hat{Y}_{n+1} = Y_n$$

$$\hat{P}_{n+1} = P_n + Q$$

Correction

$$K_{n+1} = \hat{P}_{n+1} / (\hat{P}_{n+1} + R)$$

$$Y_{n+1} = \hat{Y}_{n+1} + K_{n+1} * (H_{n+1} - \hat{Y}_{n+1})$$

$$P_{n+1} = \hat{P}_{n+1} (1 - K_{n+1})$$

Initial time estimate was taken as the **first reading of the sensor**. Sensor variance R was determined from the data. Later it was tuned to get accurate and decrease the convergence time of the filter. Initial estimation co-variance P was kept large enough for the filter to converge as fast as possible. Initially it was set to 3.

1.2.3 Sensor Calibration – Sensor was calibrated manually by taking readings of equally-spaced obstacles throughout the range of 0.1 m to 1.5 m with an interval of 0.05m. The time readings were then plotted against the distance in MATLAB. Using the **polyfit** and **polyval** commands in MATLAB. A mapping function was derived. The function was a first order equation.

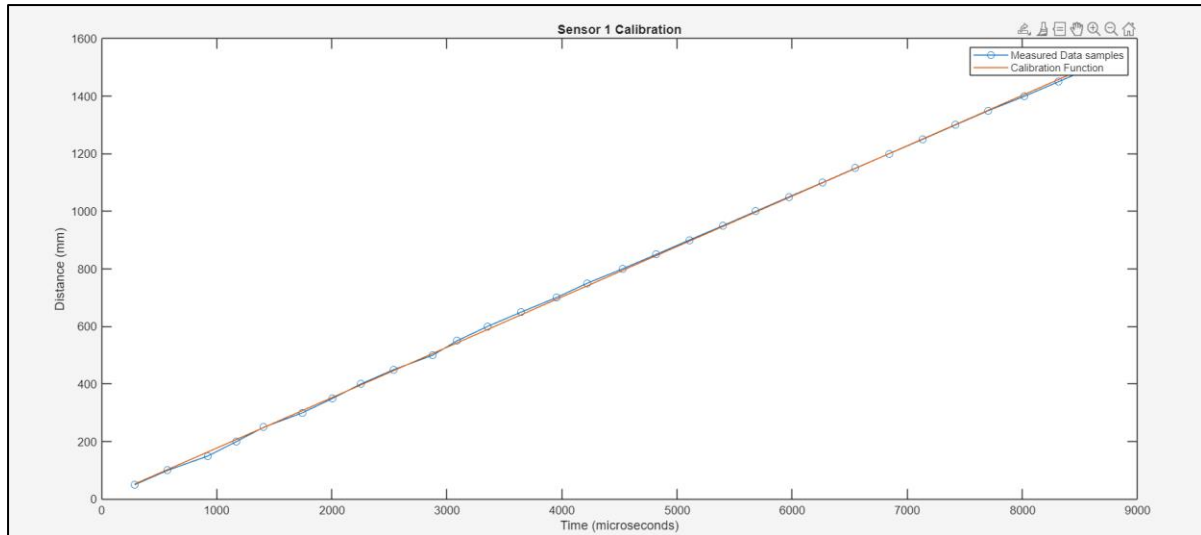


Figure 1 Calibration of Sensor 1

After tuning the constants of the function and verifying it through measurements, the calibration function was

$$\text{Distance} = (0.1752 * \text{time}) + 2.3637$$

1.3 Hardware and Software Implementation

The ultrasonic sensor was fitted to the breadboard and stuck to the RC vehicle. The sensor got supply, trigger signal and ground from the Arduino. Arduino recorded the time data from the sensor, filtered it and converted it into distance in mm. Arduino was given supply from a power bank. The code was written to stop when the error co-variance (P) value was equal to or less than a **set threshold value of 0.03**. This value was chosen because the distance value did not vary significantly after the error co-variance reached this value. In Arduino Serial Monitor we displayed the Distance and current estimated error co-variance. When the code stopped the time cost (time required for Kalman filter to converge) was also displayed.

1.4 Experimental Results

```
0.451 -> 497.65, 0.06
0.538 -> 497.83, 0.06
0.619 -> 498.00, 0.05
0.705 -> 497.83, 0.05
0.786 -> 497.65, 0.05
0.880 -> 497.48, 0.04
0.969 -> 497.13, 0.04
1.017 -> 496.95, 0.04
1.100 -> 496.78, 0.04
1.196 -> 496.60, 0.04
1.288 -> 496.43, 0.03
1.336 -> 496.60, 0.03
1.423 -> 496.43, 0.03
1.515 -> Final Distance: 496.43 mm, Time to measure: 1949 ms
```

Figure 2 Result for a distance of 500 mm

The data shows that we got an average accuracy of 99.2% and an average time cost of 1949 ms.

```

19:35:09.981 -> 394.29, 0.06
19:35:10.061 -> 394.29, 0.06
19:35:10.156 -> 394.29, 0.05
19:35:10.237 -> 394.29, 0.05
19:35:10.332 -> 394.29, 0.05
19:35:10.419 -> 394.29, 0.04
19:35:10.466 -> 394.29, 0.04
19:35:10.555 -> 394.29, 0.04
19:35:10.650 -> 394.11, 0.04
19:35:10.731 -> 393.94, 0.04
19:35:10.826 -> 393.76, 0.03
19:35:10.873 -> 393.76, 0.03
19:35:10.968 -> 393.76, 0.03
19:35:11.056 -> Final Distance: 393.76 mm, Time to measure: 1934 ms

```

Figure 3 Result for a distance of 400 mm

The data shows that we got an average accuracy of 98.44% and an average time cost of 1934 ms.

2. Sensor Fusion of Multiple Ultrasonic Sensors

2.1 Problem Statement

We were required to fuse two sensors and measure the distance of an obstacle in the range of 0.2 m to 1.5 m. The aim was to decrease the time cost and increase the accuracy of distance by using two ultrasonic sensors.

2.2 Technical Approach

2.2.1 Sampling

The sampling rate of each sensor was kept the same as earlier one. A delay of 37.5 milliseconds was kept between the triggering of two sensors to avoid signal interference.

2.2.2 Calibration

We had to first calibrate the sensor and then filter it, as the calibration function of the two sensors was different. So same distance could correspond to different time readings. Filtering this would result in less accurate results. Sensor 1 had the same calibration function as project 1.1. Sensor 2 was calibrated using the same procedure.

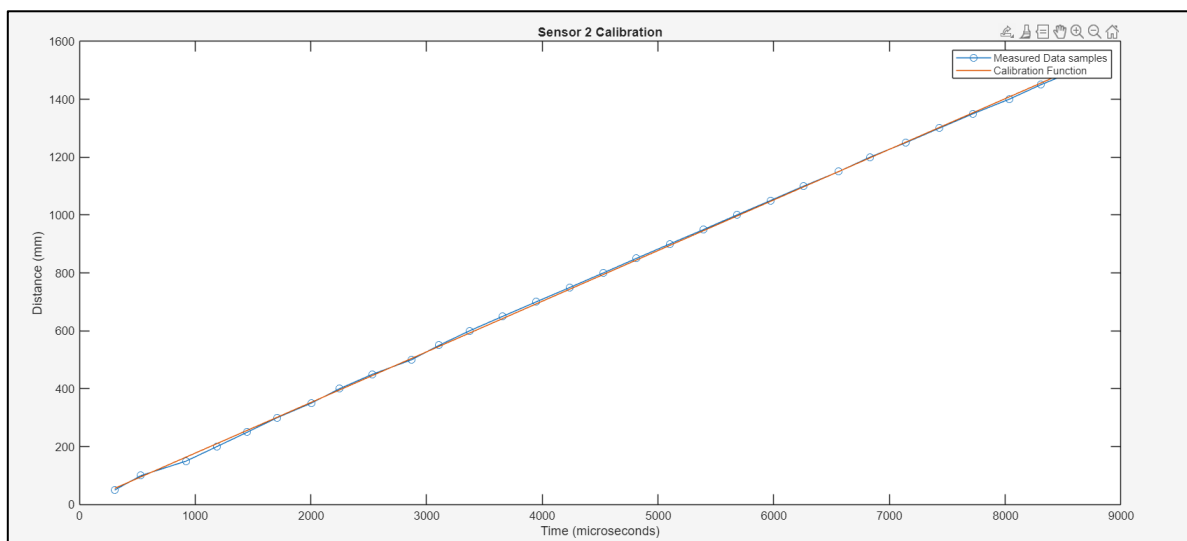


Figure 4 Sensor 2 Calibration

After tuning the constants of the function and verifying it through measurements, the calibration function for sensor 2 was, **Distance = (0.1749 * time) + 3.5889**

2.2.3 Filtering Data

For filtering the data, we used the same Kalman filter equations but this time we did the correction step twice. First time we corrected the estimate using the measurements of sensor-1 and then from the measurements of sensor-2. The filtering procedure was repeated in every loop.

For this test the variance(R) values of the two sensors were, $R_1 = 1$ and $R_2 = 0.5$. The initial estimated error co-variance (P) was kept 100. **Higher value of P and lower value of R1 and R2** were chosen so **the filter would trust the sensor readings more** and converge quickly. Initial estimated distance was taken as the first reading from sensor 1. System co-variance was taken as $Q=0$.

2.3 Hardware and Software Implementation

The two sensors were mounted on the bread board with sufficient distance between them to prevent signal interference. The breadboard was stuck on a flat surface on the vehicle facing front.

The code was designed to stop when the estimated error co-variance (P) was less than or equal to 0.03. During the test, distance and variance were displayed at each step. When the code stopped time cost was displayed.

2.4 Experimental Results

```
22.068 -> START
22.116 -> 466.00, 0.33
22.213 -> 485.00, 0.17
22.261 -> 492.00, 0.11
22.352 -> 495.00, 0.08
22.447 -> 497.00, 0.07
22.535 -> 498.00, 0.06
22.630 -> 499.00, 0.05
22.719 -> 500.00, 0.04
22.766 -> 500.00, 0.04
22.854 -> 500.00, 0.03
22.950 -> 500.00, 0.03
23.038 -> Final Distance: 500.00 mm, Time to measure: 964 ms
```

Figure 5 Result for a distance of 500 mm (fusion)

The data shows that we got an average accuracy of 100% and an average time cost of 964 ms.

```
03.833 -> START
03.874 -> 389.00, 0.33
03.971 -> 395.00, 0.17
04.058 -> 396.00, 0.11
04.155 -> 397.00, 0.08
04.200 -> 397.00, 0.07
04.295 -> 397.00, 0.06
04.384 -> 397.00, 0.05
04.479 -> 397.00, 0.04
04.527 -> 397.00, 0.04
04.613 -> 397.00, 0.03
04.709 -> 397.00, 0.03
04.797 -> Final Distance: 397.00 mm, Time to measure: 950 ms
```

Figure 6 Result for a distance of 400 mm(fusion)

The data shows that we got an average accuracy of 99.25% and an average time cost of 950 ms.

3. Conclusions and Discussions

The results represent that the accuracy of two sensors is better than that of one sensor. The time cost of the two sensors is also less than one sensor.

Time cost of project 1.2 can be decreased by taking the initial estimated distance as the average of first reading of the two sensors.

Project 2 Report: Adaptive Cruise Control and Autonomous Lane Keeping

1. Adaptive Cruise Control

1.1 Problem Statement

The aim of the project was to maintain a safe distance from a vehicle in front. To simulate this, the test required the vehicle to stop 300 mm away from the obstacle in front. This meant if the vehicle in front (obstacle) was moving at a constant speed, our vehicle would also follow the vehicle with the same speed.

1.2 Technical Approach

1.2.1 Sensors Used

We used the same two ultrasonic sensors used in project 1.1 with their respective calibration function.

1.2.2 Actuator

We had the traction motor in the RC vehicle as our main actuator. The motor required a PWM signal to operate. A PWM signal of 1500 sets the motor in neutral position. Anything above 1500 made the vehicle move ahead. The speed of the vehicle depended on the difference between the command value and 1500. Values below 1500 made the vehicle move in reverse direction. Due to the high power and torque of motor the command function was limited with a minimum value of 1250 and a maximum value of 1750. The traction motor was powered by a separate NiMH battery.

1.2.3 Vehicle Control

Non-model-based controller –

We used a Proportional Derivative Integral (PID) controller for controlling the vehicle speed. Input to the PID controller was error between desired distance to current distance of obstacle in front of the vehicle. The desired distance considering offset of sensor from the front most point of the vehicle was set to 350 mm. The PID controller would give a command to the traction motor based on the error which was added to the neutral position of motor (1500).

Tuning of the PID controller was done heuristically by trial-and-error method. All three gains were set to 0. The proportional gain of the PID controller increased in the steps of 0.1 till there were substantial oscillations in the system. Later the D-gain was increased till the oscillations were damped. If there was a steady state error (which in this case means the variation of distance of vehicle from the desired distance) then the value of I-gain was increased. The images below shows the effect of I-gain on the response of the system.

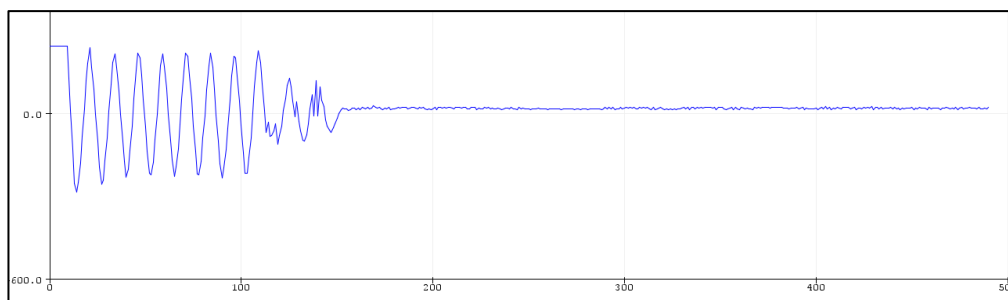


Figure 7 System response without damping (Kd)

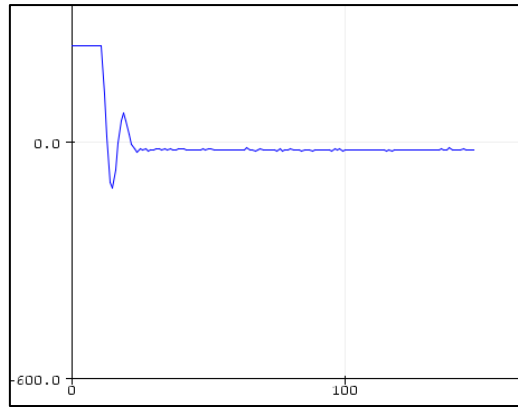


Figure 8 System response after damping (K_d)

Our final values of gains were $K_p = 1.1$, $K_i = 0$ and $K_d = 2.7$.

Model-based Controller

I tried model-based controller myself. I modelled both the cases, velocity as input and acceleration as input. I modelled the system in state-space using MATLAB. I did not implement it on the vehicle but calculated the gains and response of the system.

Velocity as input – Here, distance is the distance between vehicle and stationary obstacle, and velocity is referring to the speed of the vehicle.

$$d(\text{distance})/dt = -\text{velocity}$$

$$Y = \text{distance}$$

From Routh array we get the condition that $K < 0$.

```
%% Model based velocity as input
A_v = 0;
B_v = -1;
C_v = 1;
D_v = 0;
Q_v = 3500;
R_v = 1;
[K_v, S1_v, P1_v] = lqr(A_v, B_v, Q_v, R_v);
sys_v = ss(A_v - (B_v * K_v), B_v, C_v, D_v);
figure;
step(sys_v);
```

Figure 9 MATLAB code for velocity as input

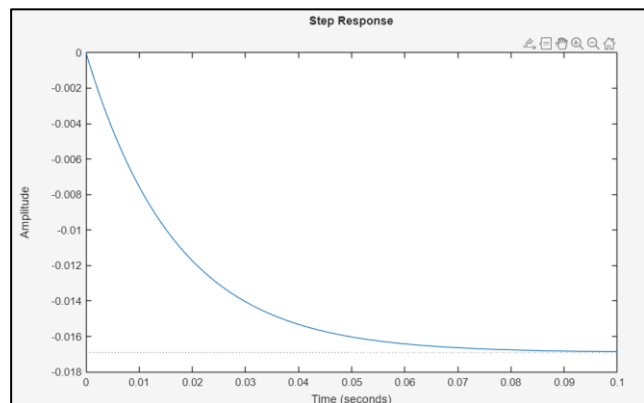


Figure 10 System response for $K = -59.16$ ($Q = 3500$ and $R = 1$)

The K is similar to the k_p of the PID. Controller input $u(t) = -K(\text{distance} - 350)$

Acceleration as input

$$d(\text{distance})/dt = -\text{velocity}$$

$$d(\text{velocity})/dt = \text{acceleration}$$

$$Y = \text{distance}$$

Routh- array condition $K_1 < 0$ and $K_2 > 0$.

```

%% Model based acceleration as input
A_a = [0,-1;0,0];
B_a = [0;1];
C_a = [1,0];
D_a = 0;
Q_a = 10000;
R_a = 1;
[K_a,S1_a,P1_a] = lqr(A_a,B_a,Q_a,R_a);
sys_a = ss(A_a-(B_a*K_a),B_a,C_a,D_a);
figure;
step(sys_a,sys_v);
legend('Acceleration as input','Velocity as input');

```

Figure 11 MATLAB code for acceleration as input

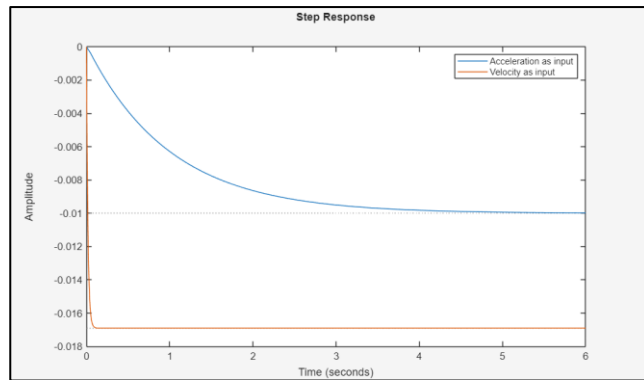


Figure 12 System response for $K_1 = -100$ and $K_2 = 100.9950$ ($Q = 10000$ and $R = 1$)

Controller input $u(t) = -K_1(\text{distance} - 350) - K_2(\text{velocity})$. Here the K_1 would be similar to the k_p in PID and K_2 to k_d in PID.

1.3 Hardware and Software Implementation

The command from Arduino was given to the traction motor. We observed that if the vehicle speed was high then the over-shoot of the vehicle increased. So, we restricted the PWM output to 1665. The sampling rate was decreased to 12.5 milliseconds to increase the speed of response.

1.4 Experimental Results

During the test our vehicle stopped almost at the desired distance from the obstacle. The PID was well tuned and damped.

2. Autonomous Lane Keeping

2.1 Problem Statement

We were required to maintain a set sideways distance from the obstacle throughout the track. The vehicle should be within the marked lines and maintain a distance of almost 350 mm from the obstacle on the left of the vehicle.

2.2 Technical Approach

2.2.1 Sensors

We used one ultrasonic sensor placed on the side of the vehicle to maintain a set distance from the obstacle.

2.2.2 Calibration

The sensor was required to be accurate within a range of 200 mm to 500 mm. So we calibrated it with more sample points in this range than in 500 mm to 1500 mm.

Calibration function for Sensor-3,

$$\text{Distance} = (0.1777 * \text{time}) - 2.4122$$

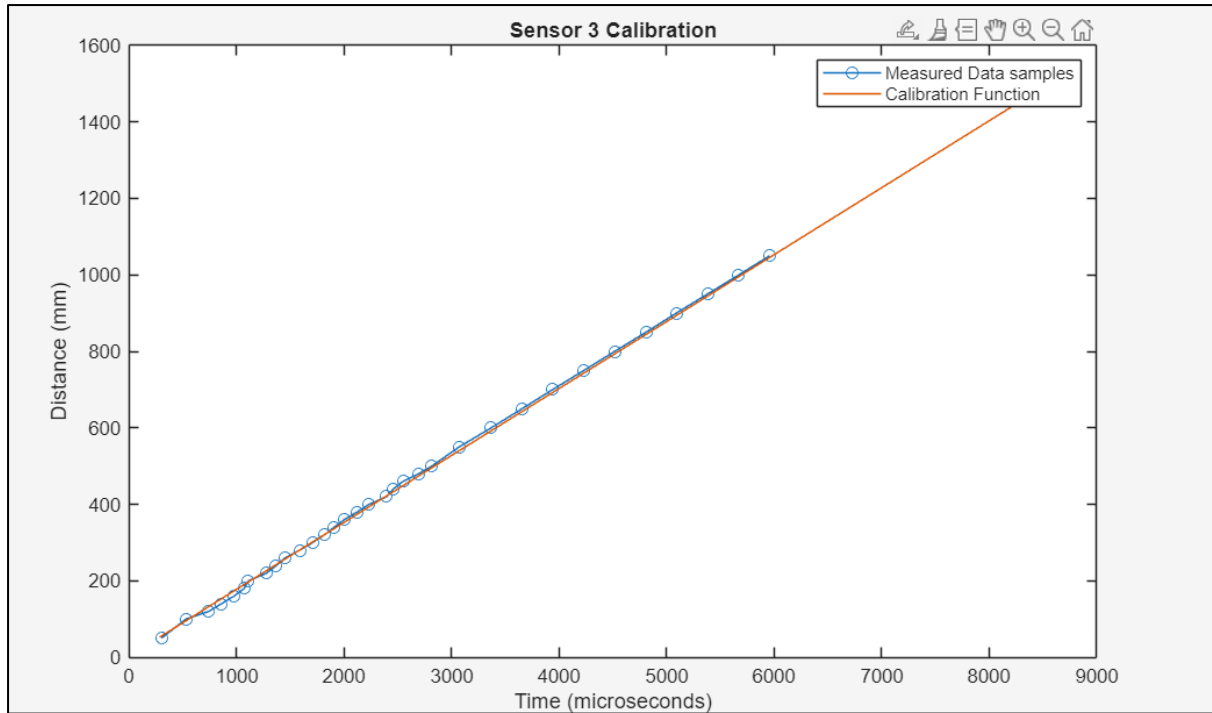


Figure 13 Calibration function for Sensor-3

2.2.3 Steering Control

We used a PID controller for the Lane keeping assist. Input to PID was the error between the desired distance and actual distance measured from the sensor-3. The PID gains were tuned in the same way as for the traction motor controller. PID gains were,

$K_p = 1$, $K_i = 0$ and $K_d = 2.5$.

2.2.4 Actuator

The actuator was a servo motor powered from the Arduino. It was also controlled by a PWM signal from Arduino. Like traction motor, neutral position of the steering was at 1500. Anything above I would turn the vehicle to right and below it to left. The maximum limit was set to 1950 and minimum was 1050.

2.3 Hardware and Software Implementation

The Servo motor was powered from the 5V output of Arduino. The response of servo motor depended on the delay of sampling and the PID gains.

2.4 Experimental Results

The tuning of steering was robust and responsive. The vehicle stayed well within the marked track and successfully maintained the desired distance.

3. Conclusions and Discussions

3.1 Conclusions

Our vehicle had a fine-tuned steering but the traction motor had some oscillation. This was mainly due to wrong sensor output. The front two sensors had some noise in its measurement due to which they gave drastically low output. Due to this the PID controller would give a corresponding low command to the traction motor. This made the vehicle stop between the track for a short time and then start again. One more problem with the sensor was that we were getting a very low (0) measurement from the sensor for an obstacle at a distance greater than 1600 mm. We tackled this by an IF loop in the code that would send a constant high measurement as an input to PID if the reading was low.

One more fault we encountered was that the input to steering motor was affecting the output of traction motor. We understood that this would be due to electrical interference between the front sensors and the steering motor as they had a common source of supply. We eliminated this by giving separate grounds to both systems.

3.2 Discussions

A major scope of improvement is the sensor filtering in Adaptive Cruise Control and Lane Keep Assist. The main faults occurred due to wrong sensor reading. We can eliminate these faults by using two different microprocessors for the steering and traction motor.

The system can be made robust by implementing a fine-tuned Kalman filter in the front two sensors as well as side sensors. This is critical because the Kalman filter needs to be optimized for quick convergence which should not induce any latency in the controller.

Feedback Response Submission

Clemson University : Survey Certificate of
Completion
202508 Fall 2025 Evaluations

Course: 202508-AUE-6220-003-91880-Jia
Instructor: Yunyi Jia

Submitted: 10/31/2025 8:25 PM
Student: Shantanu Amit More

Thank you for completing a survey! Your **feedback** is very important and greatly appreciated.